Azure AI Foundry development guide for 2025

The correct Python package for Azure AI Foundry

The actual pip package name is `azure-ai-projects`, not `azure-ai-foundry`. Microsoft uses `azure-ai-projects` as the official SDK for Azure AI Foundry functionality. Microsoft The package is currently in public preview (version 1.0.0b11 as of May 2025). PyPI Microsoft

Installation commands

```bash
# Core installation
pip install azure-ai-projects azure-identity

# Complete installation with recommended dependencies
pip install azure-ai-projects azure-identity azure-ai-inference[opentelemetry] \
            azure-search-documents azure-ai-evaluation azure-monitor-opentelemetry openai
```

Package details

- **PyPI name**: `azure-ai-projects`
- **Import statement**: `from azure.ai.projects import AIProjectClient`
- **Current version**: 1.0.0b11 (Beta/Preview) PyPI
- **Python requirements**: Python ≥3.8 (supports 3.8-3.13) PyPI
- **Status**: Public preview with regular updates

Deploying agents to Azure AI Foundry

Azure AI Foundry (the evolved successor to Azure AI Studio) provides multiple deployment approaches for AI agents. Microsoft Microsoft rebranded Azure AI Studio to Azure AI Foundry in late 2024, expanding it into a comprehensive platform for building and deploying AI agents. Prakashinfotech Microsoft

Project types for agent deployment

Foundry Projects (recommended for agents):

- Built directly on Azure AI Foundry resource Microsoft Microsoft
- Simple setup with direct agent access Microsoft Microsoft
- Best for individual developers and rapid prototyping

Hub-Based Projects:

- Enterprise-focused with advanced governance Microsoft Microsoft
- Required for certain advanced features
- Better for complex organizational deployments Microsoft

Step-by-step deployment process

1. **Create Azure AI Foundry project**:

```bash
# Via CLI
az login
az group create --name myResourceGroup --location eastus
az deployment group create \
  --resource-group myResourceGroup \
  --template-file foundry-template.json
```

2. **Deploy using Python SDK**:

```python
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential
from azure.ai.agents.models import CodeInterpreterTool

# Create client with project endpoint
project_client = AIProjectClient(
    endpoint=os.environ["PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
    api_version="latest"
)

# Create and deploy agent
agent = project_client.agents.create_agent(
    model=os.environ["MODEL_DEPLOYMENT_NAME"],
    name="my-agent",
    instructions="You are a helpful agent",
    tools=[CodeInterpreterTool().definitions]
)
```

3. **Alternative deployment methods**:

- **Portal deployment**: No-code agent builder in Azure AI Foundry portal
- **REST API**: Direct API calls for programmatic deployment
- **Container deployment**: Azure Container Apps or AKS for production scale GitHub

Endpoint format differences

Azure AI services use different endpoint formats depending on the service type and deployment model:

Azure AI Foundry project endpoints

Format: `https://<foundry-resource-name>.services.ai.azure.com/api/projects/<project-name>` Microsoft

Example:

```python
project = AIProjectClient(
    endpoint="https://myresource.services.ai.azure.com/api/projects/myproject",
    credential=DefaultAzureCredential()
)
```

Characteristics:

- Unified endpoint for all project resources Microsoft
- Project-scoped with `/api/projects/<project-name>` path Microsoft
- Uses Microsoft Entra ID authentication Microsoft Microsoft
- Best for AI agents and unified project management

Azure OpenAI endpoints

Format: `https://<resource-name>.openai.azure.com`

Example:

```python
from openai import AzureOpenAI

client = AzureOpenAI(
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    api_version="2025-04-01-preview",
    azure_endpoint="https://myresource.openai.azure.com"
)
```

Characteristics:

- Deployment-specific endpoints
- Full OpenAI API compatibility (Microsoft)
- Uses `/openai/deployments/{deployment-id}` structure
- Best for traditional OpenAI model deployments

Key differences

- **AI Foundry**: Single endpoint for multiple models and services (Microsoft)
- **Azure OpenAI**: Separate endpoints per deployment
- **Authentication**: AI Foundry prefers Microsoft Entra ID; (Microsoft) Azure OpenAI supports both API keys and Entra ID (Microsoft +2)
- **Use cases**: AI Foundry for agents and multi-model apps; Azure OpenAI for OpenAI-specific features

Current best practices for Azure AI agent deployment

Development recommendations

1. **Authentication**: Use Microsoft Entra ID with DefaultAzureCredential instead of API keys (Microsoft) (Microsoft)
2. **Project type**: Start with Foundry Projects for agent development (Microsoft) (Microsoft)
3. **Model selection**: Begin with smaller models (GPT-4o-mini) before scaling up (Microsoft)
4. **API versioning**: Use GA version (2025-05-01) for production, preview (2025-05-15-preview) for latest features (Microsoft)

Security best practices

1. **Identity management**:
   - Implement Microsoft Entra Agent ID for unique agent identities (Microsoft) (Microsoft)

- Apply conditional access policies and MFA to agents Microsoft Microsoft
- Use role-based access control (RBAC) Microsoft Microsoft

2. **Data protection**:
   - Enable content filtering with Azure AI Content Safety Microsoft +2
   - Use private endpoints and virtual networks Microsoft +2
   - Implement data residency controls for compliance
3. **Runtime security**:
   - Use Dynamic Sessions for code execution isolation Microsoft
   - Enable Microsoft Defender for Cloud integration SiliconANGLE +2
   - Implement prompt injection protection Microsoft Microsoft

Performance and cost optimization

1. **Model optimization**:
   - Use the Model Router for automatic model selection Microsoft Microsoft
   - Implement prompt caching where supported Microsoft
   - Consider batch processing for 50% cost reduction Microsoft Microsoft
2. **Resource management**:
   - Monitor costs with Azure Cost Management Microsoft
   - Use serverless deployments when appropriate
   - Implement request throttling and quotas
3. **Deployment strategy**:
   - Choose regions based on model availability and latency
   - Use multiple deployments for high availability
   - Start with standard setup for production workloads Microsoft

Recent Azure AI service changes (2024-2025)

Major rebranding and updates

1. **Azure AI Studio → Azure AI Foundry**: Complete platform rebrand with expanded agent capabilities Prakashinfotech Microsoft
2. **Agent Service GA**: Generally available as of May 2025 Microsoft Microsoft
3. **New models**: GPT-4.1 series, o3/o4-mini reasoning models, GPT-image-1 Microsoft +3
4. **Model Context Protocol (MCP)**: First-party support across Azure services Microsoft Microsoft

5. **Multi-agent orchestration**: Agent-to-Agent (A2A) communication now supported Microsoft Microsoft

Model retirements and migrations

Deprecated models:

- GPT-3.5 turbo versions 0301/0613: Retiring April 2025 Microsoft microsoft
- GPT-4 legacy versions: Retired June 6, 2025 Microsoft +2
- DALL-E 2: Deprecated January 2025 microsoft

Migration considerations:

- 60-day notice for GA model retirements Microsoft GitHub
- Test applications with new models before deadlines Microsoft GitHub
- Maintain multiple deployments during transitions Microsoft

API and SDK updates

1. **V1 Azure OpenAI APIs**: New generation APIs starting May 2025 Microsoft Microsoft
2. **Unified SDK**: Semantic Kernel and AutoGen integrated Microsoft Microsoft
3. **Enhanced monitoring**: Built-in metrics for performance, quality, cost, and safety Microsoft
4. **Response API**: New structured response capabilities Microsoft

This comprehensive transformation positions Azure AI Foundry as a production-ready platform for enterprise AI agent deployment, with significant improvements in security, cost management, and operational capabilities compared to previous Azure AI services. Microsoft Microsoft